

ESPECIFICAÇÃO ARQUITETURAL

Sistema de Gestão de Biblioteca

Disciplina: AAP IV - Programação para Internet
Atividade: Arquitetura back-end com MVC e HTTP
Aluna: Rayana Santos | Professor: Vander Elme

1. Objetivo e contexto

Este documento especifica a arquitetura proposta para acrescentar autenticação ao Sistema de Gestão de Biblioteca. Atualmente, a aplicação é composta por HTML5, CSS3 e JavaScript executados no navegador, sem back-end, banco de dados ou controle de acesso. A evolução planejada adotará o padrão Model-View-Controller (MVC) com Java Servlets e JSP, separando apresentação, controle das requisições e regras de negócio. Essa divisão facilitará manutenção, testes e a futura implementação do CRUD do acervo.

No cenário proposto, um bibliotecário acessará uma página de login para entrar na área administrativa. As credenciais serão tratadas no servidor, e somente usuários autenticados poderão acessar recursos protegidos, como o cadastro de livros e leitores. O protocolo HTTP será utilizado no ciclo de requisição e resposta; HttpSession e o cookie de sessão manterão o estado de autenticação entre requisições distintas.

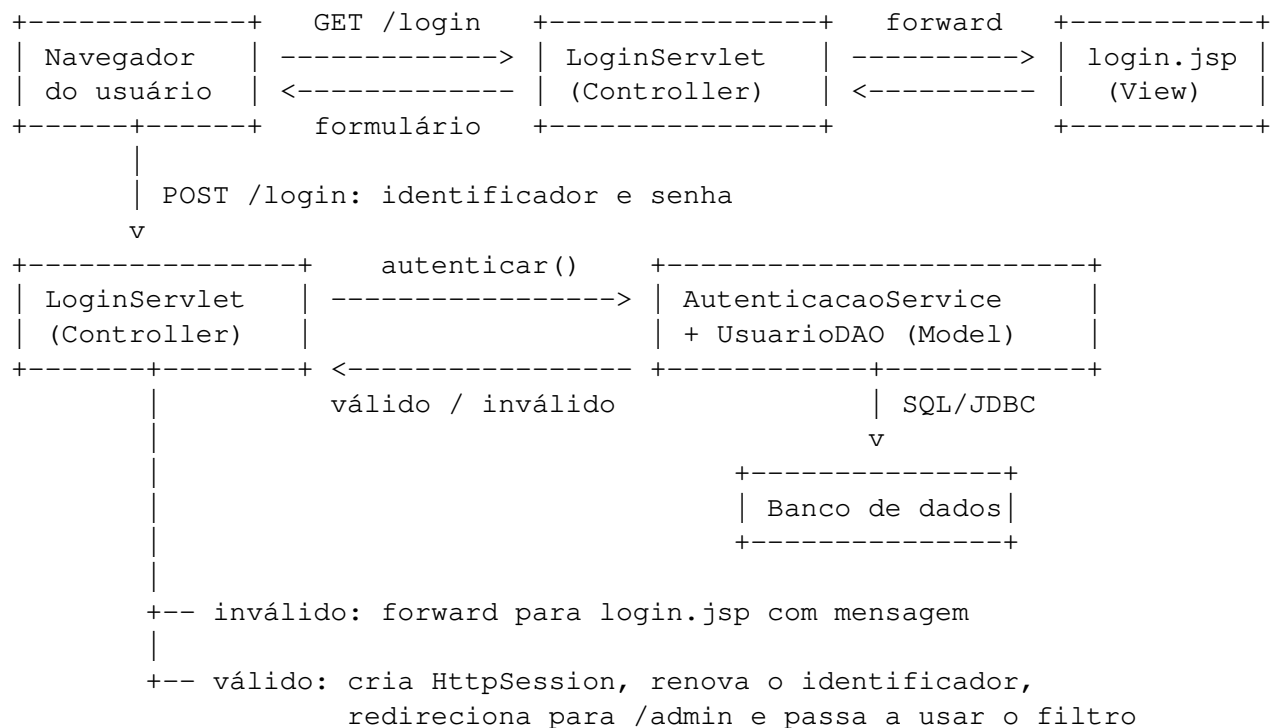
2. Componentes da arquitetura MVC

View - JSP/HTML. A View será responsável apenas pela apresentação e pela coleta de dados. login.jsp exibirá o formulário e mensagens de erro; as páginas internas mostrarão informações preparadas pelos Controllers. O formulário enviará login e senha por POST, impedindo que as credenciais apareçam na URL. As JSPs protegidas ficarão em WEB-INF/views, evitando acesso direto. A View não executará SQL, não validará senhas e não conterá regras de negócio.

Controller - Servlets e filtro. LoginServlet receberá os parâmetros e solicitará ao serviço de autenticação a conferência das credenciais. Em caso de falha, encaminhará a requisição para login.jsp com uma mensagem genérica. Em caso de sucesso, criará a sessão e redirecionará para a área administrativa. LogoutServlet invalidará a sessão. Um AuthenticationFilter interceptará URLs protegidas: permitirá a passagem quando houver sessão autenticada e, caso contrário, redirecionará ao login. Centralizar essa verificação evita código repetido e rotas internas desprotegidas.

Model - domínio, serviço e persistência. A entidade Usuario representará a identidade e o perfil; AutenticacaoService aplicará as regras; UsuarioDAO consultará o banco via JDBC. O banco armazenará um hash seguro da senha, nunca a senha em texto puro. O Model devolverá somente o resultado necessário e não dependerá das JSPs.

3. Diagrama do fluxo MVC de autenticação



3.1 Sequência de processamento

No primeiro acesso, GET /login chega ao Controller, que encaminha a requisição para a View. O usuário preenche o formulário, e POST /login entrega as credenciais ao LoginServlet. A Servlet realiza validações básicas e delega a autenticação ao Model; o serviço aplica as regras, enquanto o DAO executa uma consulta parametrizada ao banco.

Se as credenciais forem inválidas, o Controller registra uma mensagem genérica no objeto request e usa forward para reapresentar a View. A mensagem existe somente durante essa requisição. Se forem válidas, o Controller cria a HttpSession, registra a identidade mínima do usuário e redireciona para /admin. O redirecionamento após o POST impede a submissão repetida do formulário quando a página é atualizada.

Nas ações seguintes, o navegador enviará automaticamente o cookie identificador da sessão. Antes de cada Controller protegido, AuthenticationFilter consultará a sessão no servidor. Uma sessão autenticada libera o fluxo; sessão ausente, expirada ou inválida conduz novamente ao login. Após a autorização, os Controllers específicos chamam o Model e encaminham o resultado às Views, preservando o fluxo MVC em toda a aplicação.

3.2 Responsabilidades e limites

O navegador apresenta a interface e transporta dados HTTP; não decide se o usuário está autenticado. O Controller coordena, mas não consulta diretamente o banco. O Model conhece domínio, regras e persistência, mas não produz HTML. O banco persiste os usuários e seus hashes de senha. Essa delimitação possibilita substituir uma View ou uma forma de persistência com menor impacto sobre as demais partes.

4. Gestão de sessões e cookies

O HTTP é um protocolo sem estado: uma requisição não conhece, por si só, as anteriores. Conforme a abordagem de controle de estado apresentada por Basham, Sierra e Bates, a sessão associa várias requisições ao mesmo cliente sem exigir novo envio das credenciais. Após a autenticação, o servidor criará uma HttpSession e armazenará apenas identificador, nome de exibição e perfil. A senha e objetos desnecessários não serão guardados nela.

O contêiner Servlet enviará um cookie de sessão, normalmente JSESSIONID. Ele conterá um identificador opaco, e não dados pessoais. Nas requisições seguintes, o navegador devolverá o cookie; o servidor localizará a HttpSession correspondente e reconhecerá o usuário. Portanto, o estado fica no servidor, enquanto o cookie transporta somente a chave que o referencia.

4.1 Segurança, expiração e logout

O cookie será configurado com HttpOnly, reduzindo sua exposição a scripts; Secure, restringindo seu envio a HTTPS em produção; e SameSite, como proteção complementar contra requisições iniciadas por outros sites. O identificador será renovado depois do login para reduzir fixação de sessão. HTTPS será obrigatório, pois os atributos do cookie não substituem a criptografia do transporte.

A sessão terá inicialmente 20 minutos de inatividade. Esse prazo é adequado à área administrativa da Biblioteca porque limita o uso de uma sessão abandonada sem impor logins excessivamente frequentes; poderá ser revisto segundo o risco real. Depois da expiração, o filtro redirecionará ao login. No logout, LogoutServlet chamará invalidate(), encerrando imediatamente a sessão. Um cookie expirado ou sem sessão correspondente não concederá acesso.

Também serão usadas mensagens como "Usuário ou senha inválidos", sem revelar contas existentes; hash seguro de senhas; consultas parametrizadas; e autorização por perfil. A sessão comprova autenticação, mas não substitui a conferência de permissão para cada operação.

5. Parecer técnico

A sessão mantida no servidor e referenciada por cookie é apropriada porque se integra ao contêiner Servlet, evita o reenvio de credenciais e mantém informações de identidade fora do navegador. Diferentemente de guardar dados de autenticação diretamente no cookie, HttpSession permite invalidar o acesso no servidor e controlar a inatividade. Seu custo de memória é aceitável no escopo acadêmico e será limitado por poucos atributos e pelo prazo de expiração.

Conclui-se que MVC, AuthenticationFilter, HttpSession e cookie seguro oferecem separação clara de responsabilidades e controle de acesso coerente com o ciclo HTTP. A proposta evolui a interface atual para um sistema back-end sem misturar apresentação, autenticação e persistência, além de preparar a aplicação para o futuro CRUD do acervo.

Referência

BASHAM, Bryan; SIERRA, Kathy; BATES, Bert. Head First Servlets and JSP: Passing the Sun Certified Web Component Developer Exam. 2. ed. Sebastopol: O'Reilly Media, 2008.